

第 2 课 FlashAttention-2: 并行划分与非矩阵运算优化

FlashAttention v1 省去了大中间矩阵的显存往返，GPU 却仍可能因为任务太少、Warp 之间交换结果、输出反复缩放而无法充分发挥算力。FA2 保留同一个 Attention 计算目标，重新安排线程块和 Warp 的工作，并简化在线更新。本课沿这三项改动，解释它为什么更快、结果为什么不变，以及它们在 vLLM 所调用的内核中如何实现。

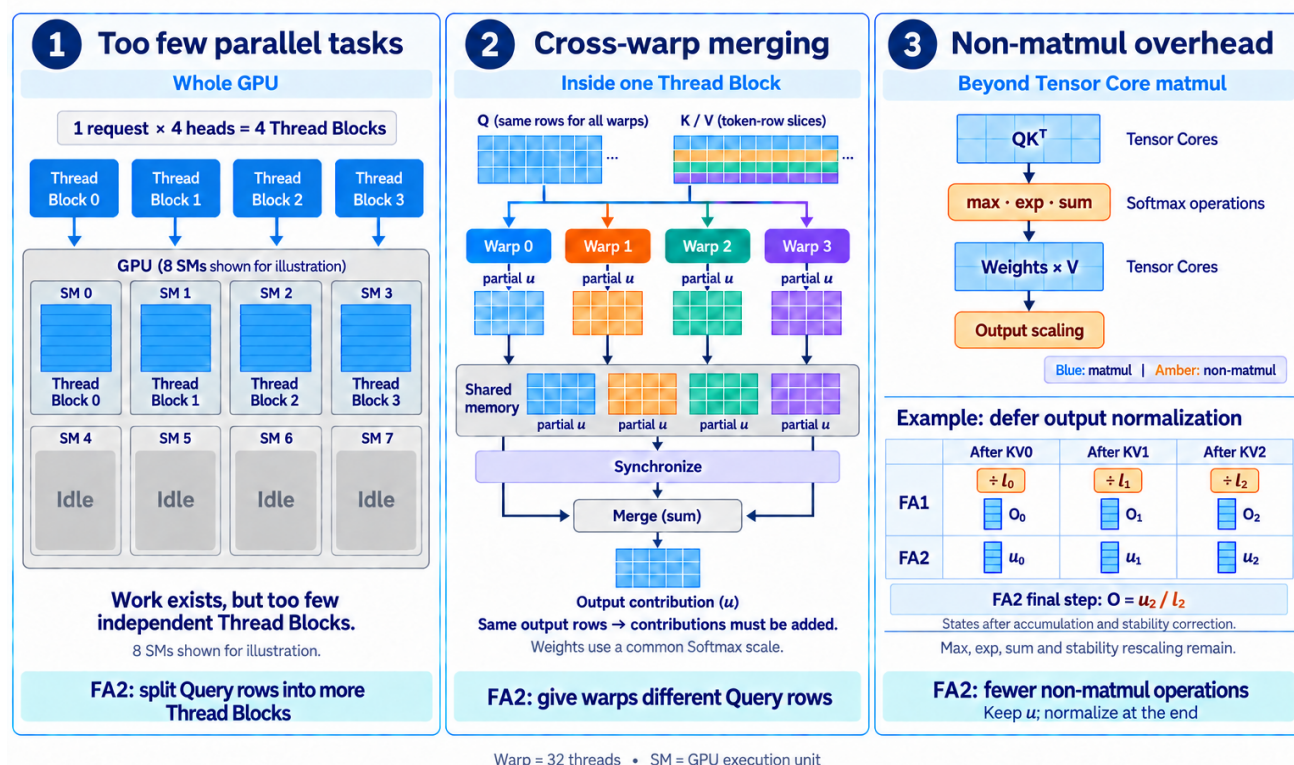
一、减少 IO 之后，还要让计算单元忙起来

FA1 已经把分数计算、Softmax 和 Value 加权融合在一起，不再把完整的分数与概率矩阵写回 HBM。接下来要解决的，是剩余工作如何在 GPU 上高效执行：

1. **任务数量**：GPU 有很多执行单元，但当前内核是否提供了足够多的独立任务？
2. **任务内部分工**：多个 Warp 算出的结果能直接写回，还是必须先交换、合并？
3. **额外计算**：矩阵乘法之外，哪些缩放与归约可以减少？

FA1: three remaining overheads

Parallel tasks → Warp communication → Non-matmul work



图右下角比较的是每轮 KV 更新后保存的状态：FA1 保存部分归一化输出 O，FA2 保存未归一化分子 u。下标表示更新轮次。

这些问题来自 FA2 论文对原始 FA1 的分析。本课讨论 forward，不含 dropout；用标准多头、无 mask 的完整 Attention 说明划分，再解释 causal mask 的影响。FA1 本来就使用 GPU 并行计算；FA2 改进的是并行维度与分工方式。

二、线程块之间：把 Query 块交给独立任务

分了数据块，为什么还会缺少并行任务

先区分三个执行层级：

对象	在本课中的作用
SM (Streaming Multiprocessor)	GPU 中执行线程块的硬件单元
线程块 (Thread Block)	一组协作线程，在一个 SM 上执行
Warp	线程块内部的执行组，由 32 个线程组成

一个 SM 在资源允许时可以同时驻留多个线程块，但一个线程块不会拆开到多个 SM 上执行。[CUDA 编程模型](#)

FA2 论文所对比的原始 FA1 实现，主要沿 batch 和 head 分配线程块。一个请求、4 个 head，只能提供 4 个这样的任务；即使每个 head 的序列很长，也可能只是让这几个任务运行更久，其他 SM 仍分不到工作。[论文 §3.2](#)

第一课已经把 Q 按行分块，但那解决的是片上空间不足。**数据切成四块，与启动四个线程块，是两件事。**前者决定一次处理多少数据，后者决定 GPU 能否把这些工作分给不同 SM。

Query 行之间没有 Softmax 依赖

每条 Query 都有自己的最大值、分母和输出。Softmax 沿 Key 方向归一化，不会把不同 Query 行放进同一个分母。因此，可以让一个线程块完整负责一组 Query 行：

$$O_i = \text{softmax}\left(\frac{Q_i K^T}{\sqrt{d}}\right) V.$$

Q_i 是第 i 个 Query 块，形状为 $B_q \times d$ ； K 、 V 均为 $N_k \times d$ ； O_i 为 $B_q \times d$ 。 B_q 是每块 Query 行数， N_k 是 Key/Value 位置数， d 是 head 维度；Softmax 对分数块的每一行独立执行。

只要负责 Q_i 的线程块遍历它所需的全部 K/V，就能独立完成 O_i 。不同线程块写入不同输出行，**最后不需要把它们的输出相加，也不需要再次做 Softmax**。读取相同 K/V 没有写入冲突。

用同一组矩阵看任务怎样变化

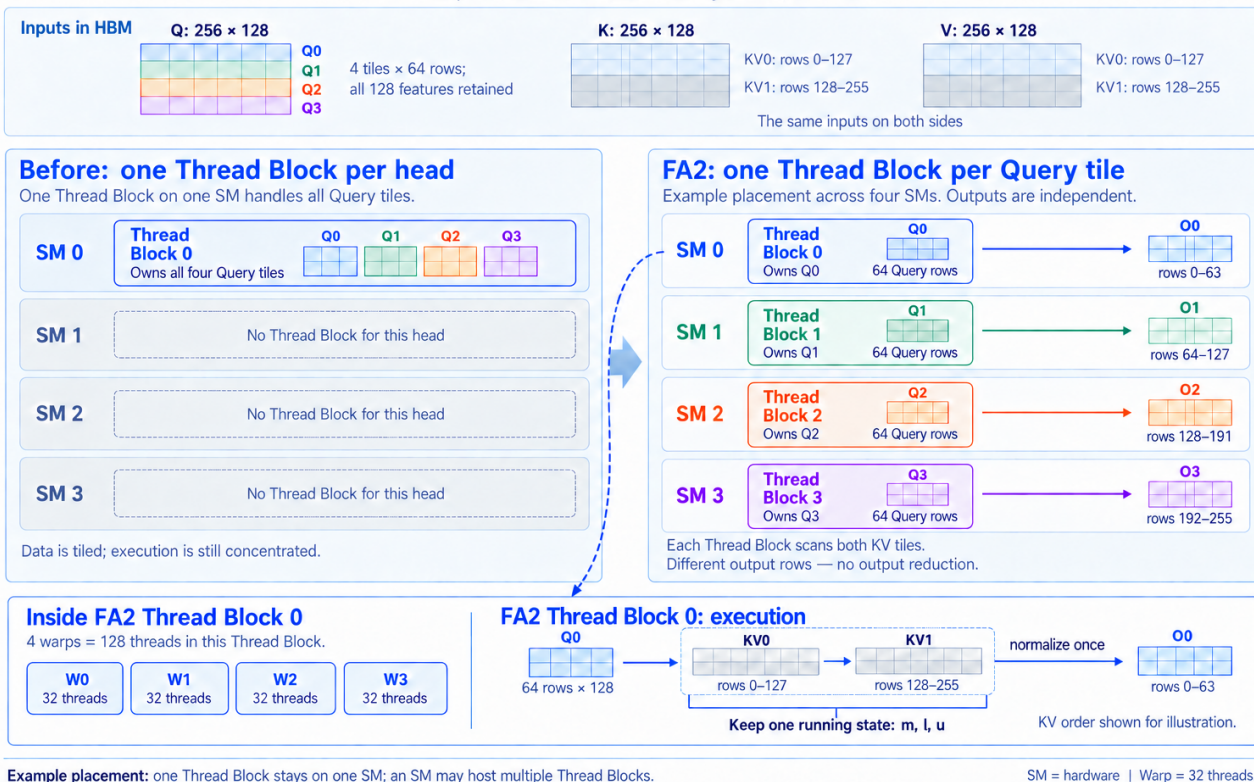
固定一个请求、一个 head：Q、K、V 均为 256×128 。Q 每 64 行分一块，K/V 每 128 行配对分一块。

对象	每块形状	块数或对应关系
Q_i	64×128	四块，各含 64 条 Query
K_j 、 V_j	各 128×128	两组，每组对应相同的 128 个位置
当前分数 S_{ij}	64×128	行是 Query，列是 Key
输出 O_i	64×128	对应 Q_i 的 64 行输出

表中 i 、 j 分别是 Query 块、KV 块编号。分块沿 token 行进行，**每个向量的 128 个特征都保留**；一个分数仍来自完整的 128 维点积。

Same Query tiles, more Thread Blocks

One request, one head | 256 Query rows | $d = 128$



图左比较原先按 head 集中分配工作的方式；图右每个线程块负责一个 Query 块，并各自扫描两组 K/V。它们分别写回输出的 0–63、64–127、128–191、192–255 行。图中把四个线程块放到四个 SM 上，只是展示一种可能的调度，不是固定绑定。

这里也改变了累计状态的存放方式。原始 FA1 Algorithm 1 外层遍历 K/V，内层遍历 Q，每轮读写各 Query 块的状态；FA2 固定一个 Query 块，在同一线程块内遍历 K/V，让该块的累计状态留在片上直到结束。**这是为了让独立输出拥有独立的执行任务，不是把 K/V 的遍历也取消了。**

第一课末尾的 vLLM Triton 示例已经采用固定 Query 块、遍历 K/V 的组织，用于说明 Online Softmax 的实际应用；它不代表原始 FA1 的线程块划分。

任务更多，不代表块越小越好

回到一个请求、4 个 head。每个 head 启动四个 Query 块任务后，线程块总数从 4 增加到：

$$1 \times 4 \times \left\lceil \frac{256}{64} \right\rceil = 16.$$

若序列有 4096 行 Query，同样每块 64 行，就能提供 256 个任务。收益是让更多 SM 有工作可做，而不是让单次点积变短。

但每个线程块都要读取它所需的 K/V。沿用 128 行 KV、 $d = 128$ ，且每个元素占 2 字节，一组 K/V 块的数据量为：

$$2 \times 128 \times 128 \times 2 = 65,536 \text{ bytes} = 64 \text{ KiB}.$$

这组数据服务 64 行 Query 时， QK^T 与指数权重乘 V 各需约 $2 \times 64 \times 128 \times 128$ FLOPs，两次合计约 4.19 MFLOPs；乘加按 2 FLOPs 计。Query 块改成 16 行后，这部分计算降为四分之一，但要加载的这组 K/V 仍为 64 KiB。

小块增加任务数，却降低了每次 K/V 装载对应的计算复用。大块提高复用，又会扩大输出累加器等片上工作集，可能因寄存器、共享内存不足而减少同时驻留的线程块。实际 HBM 流量还取决于缓存命中，不能把各任务的逻辑装载量直接当成实测流量。

加入 causal mask 后，Query 行仍独立，但任务长短不同。例如 Q 的 0–63 行只需处理第一组 KV，第二组完全不可见；128–191 行则需要两组。完全不可见的块跳过，边界块内部屏蔽未来位置。

三、线程块内部：让 Warp 负责不同输出行

现在进入图 2 的一个线程块：它处理 64 行 Query，每轮处理 128 行 K/V，由 4 个 Warp 协作。线程块之间已经独立，块内仍有两种不同的分工方式。

按 Key 分工，得到同一输出的不同贡献

如果每个 Warp 处理 32 个 Key/Value，它们都要使用相同的 64 行 Query：

每个 Warp 的工作	形状
当前分数片段	64 行 Query $\times$ 32 列 Key
对应的 Value 片段	32×128
两者加权相乘后的部分贡献	64×128

四份结果都指向同一个 64×128 输出块。虽然每个 Warp 处理不同 Key，它们只算完了同一求和的不同项，必须合并才能得到完整贡献。

设 $E_0 \cdots E_3$ 是按 Key 列切开的指数权重片段， $V_0 \cdots V_3$ 是相应的 Value 行片段；每行权重先使用共同的最大值作为指数基准，则当前 KV 块的贡献为：

$$U = E_0 V_0 + E_1 V_1 + E_2 V_2 + E_3 V_3.$$

每个 E_r 为 64×32 、 V_r 为 32×128 ， U 为 64×128 ， r 是 Warp 编号。这里不是相加四份局部 Softmax 输出；只有指数基准一致，未归一化贡献才能直接相加。

这种划分在论文中称为 **sliced-K**：此处切的是 Key 的位置列表，不是把一个 Key 的 d 个特征截短。原始实现需要把部分结果写入共享内存、同步，再读取并相加。HBM 往返减少之后，这些片上交换和等待仍有成本。[论文 §3.3](#)

按 Query 分工，结果落在不同位置

FA2 改为让四个 Warp 分别负责 16 行 Query，共同使用当前的全部 128 行 K/V。每个 Warp 的分数片段为 16×128 ，对输出的贡献也是 16×128 。

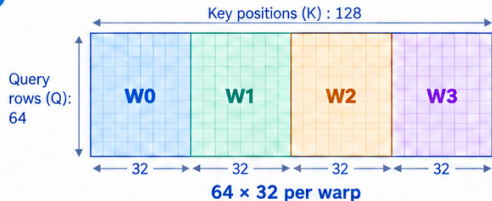
FA2: Give each warp its own output rows

One thread block • 64 Query rows • 128 Key positions • 4 warps

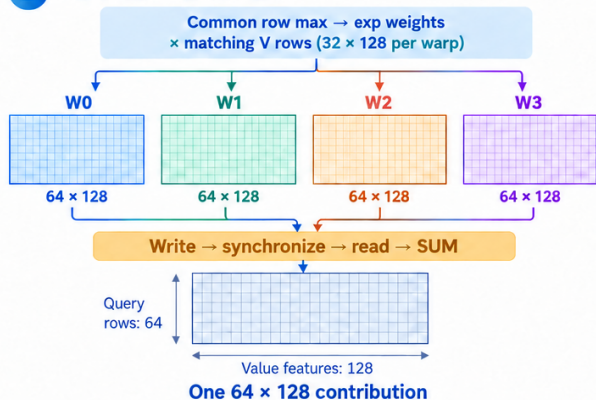
SLICED-K

Same output rows, partial contributions

1 Divide the score tile



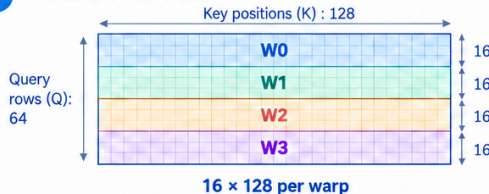
2 Form Value contributions



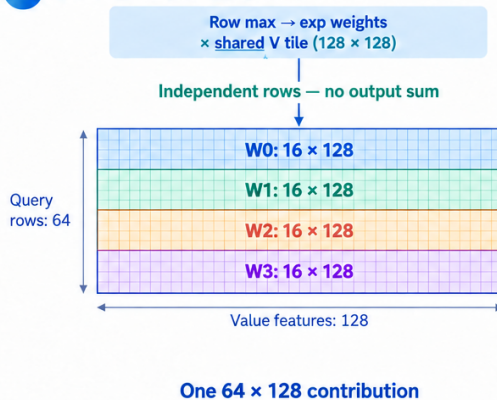
SLICED-Q

Different output rows, independent contributions

1 Divide the score tile



2 Form Value contributions



Same attention math. Shared K/V loading and within-warp reductions still remain.

This shows one KV tile's contribution, not the fully normalized result over all history.

这种 **sliced-Q** 划分利用的，仍是 Query 行之间的独立性。四份结果对应不同输出行，各自放回自己的位置即可，省掉了合并同一输出所需的跨 Warp 交换。没有少算 Query-Key 配对，也没有减少对 Value 的加权项。每个 Warp 的 16 行仍由其中的 32 个线程协作，不能理解成一个线程独自计算一整行。这里也没有消除所有同步：K/V 装载仍需协作，同一行的最大值、分母仍可能需要 Warp 内归约。FA2 的 `TiledMma` 配置正是沿矩阵行方向组织 Warp。

两个层级现在可以对应起来：线程块之间分配不同 Query 块，线程块内部再把这些 Query 行分给不同 Warp。每层都在明确“谁负责哪些输出行”，K/V 则作为这些输出共同需要的输入。

四、在线更新：把输出归一化移出 KV 循环

矩阵乘法快，为什么还要减少标量运算

QK^T 和指数权重乘 V 可以使用 Tensor Core；最大值、指数、求和、缩放等操作使用其他指令和执行资源。它们的吞吐能力不同，所以 FLOPs 少不代表耗时占比一定低。FA2 因此还要减少夹在矩阵乘法之间的非矩阵工作。论文 §3.1

其中一个可以省下的动作，是在处理完每个 KV 块后，都把当前分子转换为归一化输出。

原始 FA1 保存 O，FA2 保存 u

第一课推导 Online Softmax 时使用分子 u，随后才换成原始 FA1 Algorithm 1 保存的部分输出 O。两者满足 $u = IO$ ，区别在于循环中保存哪一个，而不是两种不同的 Attention 定义。

固定一条 Query。已经处理的历史状态为最大值 m 、指数和 l 、输出向量 O ；新块中第 j 个位置的分数为 s_j 、Value 为 v_j 。 O 、 v_j 均为 $1 \times d$ ，其余为标量。先更新共同基准：

$$\begin{aligned} m' &= \max\left(m, \max_{j \in \text{新块}} s_j\right), \\ \alpha &= e^{m-m'}, \quad e_j = e^{s_j-m'}, \\ l' &= \alpha l + \sum_{j \in \text{新块}} e_j. \end{aligned}$$

撇号表示更新后的状态， α 把旧累计量换到新最大值对应的基准， e_j 是当前块的未归一化指数权重。两种实现使用同样的最大值和分母递推。

原始 FA1 把旧输出恢复成分子，加入新贡献，再保存归一化输出：Algorithm 1，第 12 行。

$$O' = \frac{\alpha l O + \sum_{j \in \text{新块}} e_j v_j}{l'}.$$

但下一个块需要的仍然是未归一化贡献。既然最终结果才必须除以 l ，就直接保存 $1 \times d$ 的分子 $u = lO$ 。将 $u' = l'O'$ 代入上式：

$$u' = \alpha u + \sum_{j \in \text{新块}} e_j v_j, \quad O = \frac{u}{l} \quad (\text{全部可见 KV 完成后}).$$

因此，对遍历 T 个 KV 块的一条 Query，按分母缩放整个输出向量从循环内的每轮动作变成结束时的一次动作。**省掉的是中间轮次的分母归一化；最大值变化时按 α 重缩放 u 和 l ，仍然必须做。** T 表示块数，不能直接当作加速倍数。

“反复归一化”也不是重新计算历史 Key 的全部 Softmax，而是上述输出更新式中反复出现的分母因子。原始内核可以合并乘除系数，不能按公式字面断言每轮一定有独立的“先乘 l 、再除 l' ”两条指令。若一个实现原本已经保存 u ，就已经采用了这一改写。

用同一个例子核对结果与省下的动作

沿用第一课第一条 Query：四个分数为 $[0, \ln 2, \ln 4, 0]$ ，Value 依次是 $[1, 0]$ 、 $[0, 1]$ 、 $[1, 1]$ 、 $[2, 0]$ 。前两个位置为块 A，后两个为块 B。

已处理位置	共同的 m, l	FA1 循环中保存 O	FA2 循环中保存 u
A	$m = \ln 2, l = 3/2$	$[1/3, 2/3]$	$[1/2, 1]$
A 与 B	$m = \ln 4, l = 2$	$[7/8, 3/4]$	$[7/4, 3/2]$

只展开从 A 加入 B 的那一步。最大值从 $\ln 2$ 升到 $\ln 4$ ， $\alpha = 1/2$ ；两种保存方式给出的旧分子贡献分别为：

$$\begin{aligned} \text{FA1:} \quad \alpha l O &= \frac{1}{2} \times \frac{3}{2} [1/3, 2/3] = [1/4, 1/2], \\ \text{FA2:} \quad \alpha u &= \frac{1}{2} [1/2, 1] = [1/4, 1/2]. \end{aligned}$$

B 在新基准下的权重为 $[1, 1/4]$ ，贡献为 $[3/2, 1]$ 。两者加到同一个旧贡献上，都得到分子 $[7/4, 3/2]$ ；分母也同为 2，所以最终输出同为 $[7/8, 3/4]$ 。

如果 B 后面还有块，FA2 会继续保存 u ，而不是先除以 2 再继续更新。两种形式在数学上等价，浮点实现仍可能因为运算顺序和中间精度产生舍入差异。

五、把三项改动对应到 vLLM 的执行过程

一个 Query 块从输入到输出

沿用 64 行 Q、128 行 KV、4 个 Warp 的配置，FA2 的完整过程可以写成：

```
请求、head、Query 块编号共同确定一个线程块
  载入本线程块的 64 行 Q
  每行初始化  $m = -\infty$ 、 $l = 0$ 、 $u = 0$ 
  遍历全部可见 KV 块：
    协作载入当前 128 行 K/V
    各 Warp 为自己负责的 Query 行计算分数
    更新行最大值  $m$ 、指数和  $l$ ，并按新基准重缩放  $u$ 
    指数权重乘  $V$ ，把新贡献累加到  $u$ 
    复用当前分数与权重的存储
  各行计算  $O = u / l$ ，写回自己负责的输出行
```

m 、 l 各有 64 个数， u 为 64×128 。跨 KV 迭代持续更新的是这同一份状态，不是为每个 KV 块各保存一份结果、最后统一归并。

从 Python backend 到具体内核

以 vLLM v0.25.1 中已选定 FA2、使用分页 KV 的普通 Prefill 为例。标准 MHA 保持前面的 head 对应关系，不涉及特殊的 GQA Decode 重排。

入口	这一步决定什么
FlashAttentionImpl 调用 flash_attn_varlen_func	传入 Q、K/V 缓存、序列长度、页表和 fa_version
Python 接口的 fa_version == 2 分支	调用 _vllm_fa2_C.varlen_fwd
分页内核的配置分发	$d = 128$ 时，选择 64 行 Q、128 行 KV、4 个 Warp
compute_attn_splitkv 的 block 索引	Split=false 时，三个轴分别区分 Query 块、请求、head

CUDA 源码由 vLLM 构建配置固定的 flash-attention 提交引入。

这里函数名含有 splitkv，但本例没有把同一条 Query 的历史拆给多个线程块。普通分页 Prefill 分支要求 `num_splits <= 1`，进入支持分页寻址的 Split=false 路径。页表改变 K/V 地址的查找方式，不改变本例对输出行的分工。

在线状态对应哪些变量

课中的动作	内核中的对应实现
更新 m 、 l ，修正旧 u	<code>softmax_rescale_o(acc_s, acc_o, ...)</code> ：分数转为指数权重，旧 <code>acc_o</code> 按新基准缩放
累加当前块的 Value 贡献	<code>gemm_rs(acc_o, ...)</code> ： <code>acc_o</code> 持续保存分子 u
KV 循环结束，得到 O	<code>normalize_softmax_lse</code> ：按行分母缩放 <code>acc_o</code>

`softmax_rescale_o` 内部用 `row_max` 维护最大值，用 `row_sum` 累计指数和；通过 `exp2` 和合入的缩放系数实现同样的稳定指数更新。

延后归一化还提供了一个具体的执行机会：循环中不必立即使用完整分母，可以先让线程保留各自的局部 `row_sum`，到 `normalize_softmax_lse` 才进行同行所需的线程间求和、取倒数并缩放输出。这里推迟的是 **Warp 内分母的归约**，与第三节消除跨 Warp 的部分输出合并是两件事。

教材按 A、B 顺序便于手算；该内核实际上从后向前遍历 KV 块，先处理需要 `mask` 的边界。只要可见块各处理一次、累计量使用一致基准，数学结果不依赖遍历方向。

六、收益取决于原来卡在哪里

FA2 的改动没有减少 dense Attention 所需的 Query-Key 配对数。它让已有工作更容易分散到 GPU 上执行，并减少协作和非矩阵操作的成本；因此，收益应结合输入形状判断。

场景	能从三项改动中获得什么
Query 很多、batch/head 较少的长 Prefill	Query 分块能增加独立任务，改善 SM 利用
原本已有大量 batch/head 任务	新增 Query 并行的边际收益可能降低，块内分工和缩放仍有优化空间
普通单 token Decode	每请求、每 head 只有一行 Query，无法沿这一维再拆；表现更多取决于 KV 读取与其他并行维度

论文 §4.1、图 4 在 A100 80GB SXM4 上测试了序列长度 512–16K、head 维度 64/128 和不同因果设置，前向加反向的合计性能相对原始 FA 约提升到两倍。这个数字包含反向，不能当作 vLLM 推理的固定加速比。在推理服务中，应分别看 Attention 内核耗时、整次模型前向耗时和请求吞吐。FA2 直接改善的是前者；最终收益还受模型其他算子、KV 访问和服务负载限制。